Generative AI Curriculum

Module 9: Interacting with Data Using LangChain and RAG

Ultra-Detailed Guide to Retrieval-Augmented Generation

INTRODUCTION TO RAG

RAG = Retrieval-Augmented Generation

Core Idea: Combine LLM's knowledge with external data for accurate, up-to-date responses

Problem It Solves:

- **Hallucinations**: LLMs generate false information
- **Outdated Knowledge**: Training data becomes stale
- **Domain-Specific Queries**: LLM lacks specialized knowledge
- **Verifiability**: No way to trace where information came from

THE RAG PROBLEM

Without RAG: Hallucination Problem

```
User: "What was Apple's revenue in Q3 2024?"
LLM: "Based on my training data from April 2024,
      I estimate approximately $90 billion"
      (Training data outdated, estimate wrong)

Actual Q3 2024 Revenue: $93.73 billion

Error: Model hallucinated/estimated incorrectly
```

With RAG: Accurate, Sourced Answer

```
User: "What was Apple's revenue in Q3 2024?"

RAG System:
1. Retrieve: Search knowledge base for Apple Q3 2024
2. Find: Document with official earnings report
3. Augment: Add retrieved document to prompt
4. Generate: LLM uses document as source

Response: "According to Apple's official Q3 2024
```

```
          earnings report, revenue was $93.73 billion
          [Source: apple.com/newsroom]"

Accuracy: 100% (directly from source)
Verifiable: User can check source
```

PART 1: RAG ARCHITECTURE

The RAG Pipeline

Step 1: Document Loading

Input: Raw documents (PDFs, websites, databases)

```
Document Sources:
  ├─ PDFs (user manuals, reports)
  ├─ Websites (company info, FAQs)
  ├─ Databases (customer data, product info)
  └─ APIs (real-time data)

Processing:
  Raw documents → Extract text → Clean
```

Step 2: Document Splitting

Challenge: Large documents too long for context windows

```
Document: 100-page manual
LLM Context Window: 4,096 tokens

Solution: Split into smaller chunks

Chunk 1: "Pages 1-3 (Installation)"
Chunk 2: "Pages 4-10 (Basic Usage)"
Chunk 3: "Pages 11-20 (Advanced Features)"
...

Chunk Size: typically 300-1000 tokens
Overlap: 100-200 tokens (preserve continuity)
```

Code Example:

```
from langchain.text_splitter import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=100
)
```

```
chunks = splitter.split_documents(documents)
```

Step 3: Embedding Generation

Purpose: Convert text to vectors for similarity search

Process:

```
Chunk: "The installation requires 20 minutes"
  ↓ Embedding Model
Vector: [0.2, 0.5, -0.1, 0.8, ..., 0.3]  (384-dimensional)
  ↓ Store in Vector Database
```

Why Embedding?:

- Numerical representation enables similarity search
- Similar meanings → nearby vectors
- Fast retrieval using distance metrics

Step 4: Vector Store

Storage: Index of document chunks and embeddings

```
Vector Store Structure:
┌──────────┬──────────────────────────┬──────────────────────┐
│ Chunk ID│ Embedding Vector          │ Original Text      │
├──────────┼──────────────────────────┼──────────────────────┤
│ Doc-001 │ [0.2, 0.5, -0.1, ...]   │ "Installation..."  │
│ Doc-002 │ [0.3, 0.4, 0.2, ...]    │ "Troubleshooting..." │
│ Doc-003 │ [0.1, 0.8, -0.3, ...]   │ "FAQ..."           │
└──────────┴──────────────────────────┴──────────────────────┘

Vector Databases:
  - Pinecone (cloud)
  - Weaviate (self-hosted)
  - Faiss (fast, local)
  - Milvus (scalable)
  - Chroma (simple)
```

Step 5: Retrieval

At Query Time: Find most relevant chunks

```
User Query: "How do I install this?"
  ↓ Embed query to same space
Query Vector: [0.25, 0.45, -0.05, 0.75, ..., 0.35]
  ↓ Search vector store (cosine similarity)
Most Similar Chunks:
```

```
  1. "Installation steps..." (Similarity: 0.92)
  2. "Quick start guide..." (Similarity: 0.87)
  3. "System requirements..." (Similarity: 0.81)
  ↓ Return top-k chunks (e.g., k=3)
```

Similarity Calculation:

$$\text{Similarity} = \frac{u \cdot v}{||u|| \cdot ||v||}$$

Where u and v are embedding vectors (cosine distance)

Step 6: Generation with Context

Augment Prompt with Retrieved Chunks

```
System: You are a helpful assistant

Retrieved Context:
"Installation requires:
  1. Download software from website
  2. Run installer
  3. Follow on-screen prompts
  4. Restart computer
  Takes approximately 20 minutes"

User Query: "How do I install this?"

Model Response: "Follow these steps...
               [generates using context]"
```

PART 2: DOCUMENT PROCESSING

Loading Different Document Types

Text Files

```
from langchain.document_loaders import TextLoader

loader = TextLoader("document.txt")
documents = loader.load()
```

PDFs

```
from langchain.document_loaders import PyPDFLoader

loader = PyPDFLoader("document.pdf")
documents = loader.load()
```

```
# Preserves structure, handles multi-page
```

Web Pages

```
from langchain.document_loaders import WebBaseLoader

loader = WebBaseLoader("https://example.com")
documents = loader.load()
```

Databases

```
from langchain.document_loaders import SQLDatabaseLoader

loader = SQLDatabaseLoader(
    uri="postgresql://...",
    query="SELECT * FROM documents"
)
documents = loader.load()
```

Splitting Strategies

Fixed Chunk Size

```
Chunk Size: 500 tokens
Overlap: 100 tokens

Result: Regular, predictable chunks
Issue: Might split mid-sentence
```

Semantic Splitting

```
Analyze document structure:
  - Chapters
  - Sections
  - Paragraphs

Split at semantic boundaries

Result: Coherent, meaningful chunks
Issue: Requires more processing
```

PART 3: VECTOR STORES AND EMBEDDINGS

Embedding Models

OpenAI Embeddings

```
from langchain.embeddings import OpenAIEmbeddings

embeddings = OpenAIEmbeddings(model="text-embedding-3-large")

# Dimension: 3,072
# Cost: ~$0.02 per 1M tokens
# Quality: Excellent
```

Open Source Embeddings

```
from langchain.embeddings import HuggingFaceEmbeddings

embeddings = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-mpnet-base-v2"
)

# Dimension: 768
# Cost: Free (self-hosted)
# Quality: Good
# Speed: Fast
```

Vector Store Integration

Storing Embeddings

```
from langchain.vectorstores import Chroma

vector_store = Chroma.from_documents(
    documents=chunks,
    embedding=embeddings,
    persist_directory="./chroma_db"
)

# Automatically embeds and stores all chunks
```

Similarity Search

```
# Query vector store
query = "How do I troubleshoot errors?"

results = vector_store.similarity_search(query, k=3)
```

```
for result in results:
    print(f"Similarity: {result.metadata['score']}")
    print(f"Content: {result.page_content}")
```

Hybrid Search

Combine Multiple Retrieval Methods:

```
Pure Vector Search:
  Strength: Semantic understanding
  Weakness: Misses exact keyword matches

BM25 Keyword Search:
  Strength: Exact phrase matching
  Weakness: No semantic understanding

Hybrid Approach:
  1. Vector search: Top 10 semantically similar
  2. BM25 search: Top 10 keyword matches
  3. Ensemble: Combine and rerank
  Result: Best of both worlds
```

PART 4: RETRIEVAL TECHNIQUES

Basic Retrieval

Similarity-Based:

```
Query embedding → Find nearest neighbors → Return chunks
```

Advanced Retrieval Techniques

1. Reranking

Problem: Retrieved chunks might not be perfectly ordered

```
Initial Ranking (by similarity):
  1. "Installation" (Similarity: 0.92)
  2. "Troubleshooting" (Similarity: 0.87)
  3. "FAQ" (Similarity: 0.81)

Reranking (by relevance):
  1. "Installation" (Score: 0.98)
  2. "FAQ" (Score: 0.94)
  3. "Troubleshooting" (Score: 0.85)

Better: Top result now most relevant
```

2. Recursive Retrieval

Handle Multi-Level Documents:

```
Query: "Advanced features"
  ↓ First Retrieval
  Get: Table of Contents entry
  ↓ Parse ToC
  Identify: Chapter 8 is "Advanced Features"
  ↓ Second Retrieval
  Get: Chapter 8 content
  ↓ Return: Detailed information from specific chapter
```

3. Multi-Query Retrieval

Query Expansion:

```
Original Query: "How to install?"

Generated Variations:
  1. "Installation steps"
  2. "Setup instructions"
  3. "Getting started"
  4. "System requirements"

Retrieve for Each:
  Results 1, 2, 3, 4

Combine Results:
  Comprehensive coverage
```

PART 5: QUESTION-ANSWERING SYSTEMS

Basic QA Pipeline

```
User Question: "What are the system requirements?"
  ↓
Retriever (Vector Store):
  Get relevant documentation chunks
  ↓
Chunks Retrieved:
  "Processor: Intel i5 or AMD Ryzen 5"
  "RAM: 8GB minimum"
  "Storage: 50GB SSD"
  ↓
Generator (LLM):
  Synthesize chunks into answer
  ↓
```

```
Answer: "System requirements are...
         [synthesized from chunks]"
```

Building QA Systems

Simple QA

```
from langchain.chat_models import ChatOpenAI
from langchain.vectorstores import Chroma
from langchain.embeddings import OpenAIEmbeddings

# Setup
embeddings = OpenAIEmbeddings()
vector_store = Chroma.from_documents(docs, embeddings)
llm = ChatOpenAI(model="gpt-4")

# Query
query = "How do I install?"
retrieved_docs = vector_store.similarity_search(query, k=3)

# Format prompt with context
context = "\n".join([doc.page_content for doc in retrieved_docs])
prompt = f"""
Context:
{context}

Question: {query}

Answer:
"""

# Generate
response = llm.predict(prompt)
```

Evaluation Metrics

Retrieval Quality

Metrics:

```
Precision: What fraction of retrieved docs are relevant?
Recall: What fraction of relevant docs are retrieved?
NDCG (Normalized Discounted Cumulative Gain):
  Is most relevant doc ranked first?
```

Generation Quality

Metrics:

```
BLEU: Overlap with reference answer
ROUGE: Summary quality
Human Evaluation: Does answer satisfy human?
Factuality: Is answer factually correct?
```

PART 6: BUILDING RAG MODELS USING LANGCHAIN

LangChain Basics

LangChain: Framework for building LLM applications

```
from langchain.chat_models import ChatOpenAI
from langchain.vectorstores import Chroma
from langchain.embeddings import OpenAIEmbeddings
from langchain.chains import RetrievalQA

# Initialize components
embeddings = OpenAIEmbeddings()
vector_store = Chroma.from_documents(documents, embeddings)
llm = ChatOpenAI(model="gpt-4")

# Create RAG chain
qa_chain = RetrievalQA.from_chain_type(
    llm=llm,
    chain_type="stuff",  # Combine all docs at once
    retriever=vector_store.as_retriever()
)

# Query
response = qa_chain.run("How do I install?")
```

Chain Types

Stuff

Combine all retrieved docs into one prompt

```
Prompt:
"Context:
[all_retrieved_chunks_combined]

Question: [query]
Answer:"

Use: When total tokens &lt; max context
```

```
Advantage: Simple, fast
Disadvantage: May exceed token limit
```

Map-Reduce

Process each doc separately, then summarize

```
Step 1: Process each chunk
  Chunk 1 → Answer 1
  Chunk 2 → Answer 2
  Chunk 3 → Answer 3

Step 2: Combine answers
  [Answer1, Answer2, Answer3] → Final Answer

Use: Many large documents
Advantage: Handles long documents
Disadvantage: More API calls, slower
```

Refine

Iteratively refine answer

```
Step 1: Get answer from first chunk
  Answer = "A"

Step 2: Refine with second chunk
  Input: "Previous answer: A. New info from chunk2"
  Output: "Refined answer: B"

Step 3: Further refine with third chunk
  Input: "Previous answer: B. New info from chunk3"
  Output: "Final answer: C"

Use: Building comprehensive answer
Advantage: High quality
Disadvantage: Many API calls
```

Building Complete RAG System

```
from langchain.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.vectorstores import Pinecone
from langchain.embeddings import OpenAIEmbeddings
from langchain.chat_models import ChatOpenAI
from langchain.chains import RetrievalQA

# Step 1: Load documents
loader = PyPDFLoader("manual.pdf")
documents = loader.load()
```

```
# Step 2: Split into chunks
text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=200
)
chunks = text_splitter.split_documents(documents)

# Step 3: Create embeddings and store
embeddings = OpenAIEmbeddings()
vector_store = Pinecone.from_documents(
    chunks,
    embeddings,
    index_name="my-index"
)

# Step 4: Create QA chain
llm = ChatOpenAI(model="gpt-4", temperature=0)
qa_chain = RetrievalQA.from_chain_type(
    llm=llm,
    chain_type="stuff",
    retriever=vector_store.as_retriever(search_kwargs={"k": 3})
)

# Step 5: Query
response = qa_chain.run("How do I troubleshoot errors?")
print(response)
```

ADVANCED RAG TECHNIQUES

1. Metadata Filtering

Filter retrieved docs by metadata

```
# During indexing
metadata = {
    "source": "FAQ",
    "date": "2024-01-15",
    "category": "installation"
}

# During retrieval
results = vector_store.similarity_search(
    "How to install?",
    k=3,
    filter={"source": "FAQ"}  # Only FAQ documents
)
```

2. Hybrid Search

Combine vector and keyword search

```
from langchain.retrievers import BM25Retriever, EnsembleRetriever

# Vector search
vector_retriever = vector_store.as_retriever(k=5)

# Keyword search (BM25)
bm25_retriever = BM25Retriever.from_documents(chunks)

# Ensemble
ensemble_retriever = EnsembleRetriever(
    retrievers=[vector_retriever, bm25_retriever],
    weights=[0.6, 0.4]
)

# Best of both worlds
results = ensemble_retriever.get_relevant_documents("query")
```

3. Multi-Hop Retrieval

Retrieve → Answer Question → Retrieve Again

```
Query: "How do I install and then configure?"

Hop 1: Retrieve installation docs
       Answer: "Installation steps"

Hop 2: Retrieve configuration docs
       Answer: "Configuration steps"

Result: Comprehensive multi-step answer
```

REFERENCES

1. Lewis, P., Perez, E., Piktus, A., Schwenk, H., Schwab, D., Kiela, D., & Grave, E. (2023). Retrieval-augmented generation for knowledge-intensive NLP tasks. In Advances in Neural Information Processing Systems.
2. Karpukhin, V., Oguz, B., Min, S., Lewis, P., Wu, L., Edunov, S., ... & Schwenk, H. (2023). Dense passage retrieval for open-domain question answering. In Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP).
3. Guu, K., Lee, K., Tung, Z., Pasupat, P., & Chang, M. (2023). REALM: Retrieval-augmented language model pre-training. In International Conference on Machine Learning (pp. 3929-3938).
4. Tam, D., Masry, A., Joty, S., & Saphra, N. (2023). Improving and simplifying pattern filtering for relation extraction. In Findings of the Association for Computational Linguistics: ACL 2023 (pp. 5980-5995).

5. LangChain Documentation. (2024). Retrieved from https://docs.langchain.com/
6. Prompting Guide - RAG. (2023). Retrieved from https://www.promptingguide.ai/research/rag

PRACTICAL EXERCISE

Build Your Own RAG System

```
# 1. Prepare your documents
docs = load_your_documents()  # PDF, website, database

# 2. Split into chunks
chunks = split_documents(docs, chunk_size=500)

# 3. Create embeddings
embeddings = create_embeddings()

# 4. Store in vector database
vector_store = create_vector_store(chunks, embeddings)

# 5. Create QA system
qa_system = build_qa_chain(vector_store)

# 6. Query
answer = qa_system.query("Your question here?")
```

Next Steps

The next module covers **LangChain for Application Development**, where you'll learn to:

- Build complete applications with LangChain
- Integrate multiple components
- Deploy to production
- Handle errors and edge cases

[1] [2] [3] [4] [5] [6] [7] [8] [9] [10] [11] [12] [13] [14] [15] [16] [17] [18] [19] [20]

⁂

1. https://ai.google/principles/
2. https://arxiv.org/abs/2312.10997
3. https://help.openai.com/en/articles/6654000-best-practices-for-prompt-engineering-with-the-openai-api
4. https://www.promptingguide.ai/techniques
5. https://www.k2view.com/blog/prompt-engineering-techniques/
6. https://aws.amazon.com/blogs/machine-learning/prompt-engineering-techniques-and-best-practices-learn-by-doing-with-anthropics-claude-3-on-amazon-bedrock/
7. https://arxiv.org/html/2507.03405v1
8. https://ubiai.tools/the-complete-guide-to-generative-adversarial-networks-gans/

9. https://arxiv.org/html/2411.14511v1
10. https://dextralabs.com/blog/prompt-engineering-for-llm/
11. https://www.artificialintelligencepub.com/jairi/article/view/jairi-aid1004/5887
12. https://dl.acm.org/doi/10.1145/3613904.3642466
13. https://www.sciencedirect.com/science/article/pii/S2666827024000483
14. https://www.ibm.com/think/topics/generative-ai
15. https://aiverifyfoundation.sg/wp-content/uploads/2024/05/Model-AI-Governance-Framework-for-Generative-AI-May-2024-1-1.pdf
16. https://www.centraleyes.com/generative-ai-governance/
17. https://www.geeksforgeeks.org/nlp/transformer-attention-mechanism-in-nlp/
18. https://www.promptingguide.ai/research/rag
19. https://www.informationpolicycentre.com/uploads/5/7/1/0/57104281/cipl_applying_data_protection_principles_genai_dec24.pdf
20. https://arxiv.org/abs/1706.03762